SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

SW문제해결 기본

List 2

Python

리스트 안에서 기준보다 큰 값을 찾아 개수를 세어 봅시다!

1. 숫자들이 들어 있는 리스트를 변수 [ ] 에 저장합니다.
2. 반복문을 이용해 리스트의 값을 하나씩 꺼내면서 [ ] 조건을 사용해 50보다 큰 값만 골라냅니다.
3. 조건을 만족하면 [ ] 값을 1씩 증가시켜 개수를 셉니다.

- 리스트 안의 여러 값을 반복문으로 하나씩 꺼내 처리할 수 있어요.
- 조건문을 통해 특정 기준(예: 50보다 큰 값)을 만족하는 값만 필터링할 수 있어요.
- 조건을 만족할 때마다 count += 1 을 통해 누적 개수를 셀 수 있습니다.
- 이 구조는 데이터 분석에서 조건 필터링 시 자주 활용됩니다.

학습목표

- 2차원 리스트를 선언하고 활용할 수 있다.
- 2차원 리스트의 순회 방법을 이해하고 활용할 수 있다.
- 부분집합을 구하는 알고리즘을 이해하고 활용할 수 있다.
- 검색의 개념을 설명할 수 있다.
- 인덱스 개념을 이해하고 활용할 수 있다.
- 셀렉션 알고리즘에 대하여 설명할 수 있다.
- 선택 정렬을 이해하고 활용할 수 있다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

2차원 List

- 2차원 배열
- 델타
- 연습문제
- 부분집합

검색과 정렬

- 순차검색
- 이진 검색 (Binary Search)
- 선택 정렬 (Selection Sort)
- 셀렉션 알고리즘 (Selection Algorithm)

2차원 List

2차원 배열

2차원 배열의 선언

- 1차원 List를 묶어놓은 List
- 2차원 이상의 다차원 List는 차원에 따라 Index를 선언
- 2차원 List의 선언 : 세로길이(행의 개수), 가로길이(열의 개수)를 필요로 함
- Python 에서는 데이터 초기화를 통해 변수 선언과 초기화가 가능함

※ arr = [[0,1,2,3],[4,5,6,7]] (2행 4열의 2차원 List)

arr	0	1	2	3	열
0	0	1	2	3	
1	4	5	6	7	
행					

<그림1 _SW문제해결 기본_List2_ 2차원 List>

입력을 2차원 배열에 저장하기

※ 입력 예시입니다

```
3
1 2 3
4 5 6
7 8 9
```

```
N = int(input)
arr = [list(map(int, input().split())) for _ in range(N)]
```

```
3
123
456
789
```

```
N = int(input)
arr = [list(map(int, input())) for _ in range(N)]
```

0으로 채워진 3x4 배열 만들기

	0	1	2	3
0	0	0	0	0
1	0	0	0	0
2	0	0	0	0

<그림2 _SW문제해결 기본_List2_ 0으로 채워진 3x4 배열 >

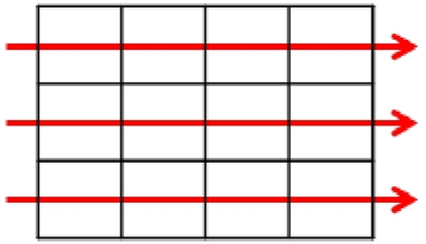
```
arr = [[0] * 4 for _ in range(3)]
```

배열 순회

- n X m 배열의 n*m 개의 모든 원소를 빠짐없이 조사하는 방법

행 우선 순회

```
# i 행의 좌표
# j 열의 좌표

for i in range(n):
    for j in range(m):
        f(array[i][j]) #필요한 연산 수행
```



행 우선 순회

<그림3 _SW문제해결 기본_List2_ 행 우선 순회>

N x M 배열의 크기와 저장된 값이 주어질 때 합을 구하는 방법

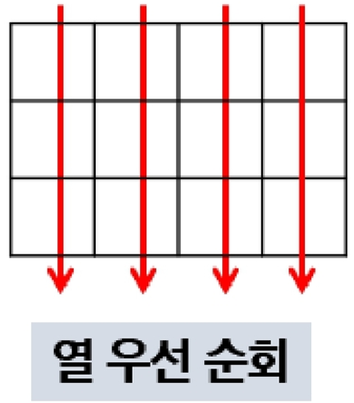
입력 예시입니다

```
3 4
1 7 2 8
6 2 9 3
5 7 4 2
```

```
N, M = map(int, input().split())
arr = [list(map(int, input().split())) for _ in range(N)]

s = 0

for i in range(N):
    for j in range(M):
        s += arr[i][j]
```

열 우선 순회

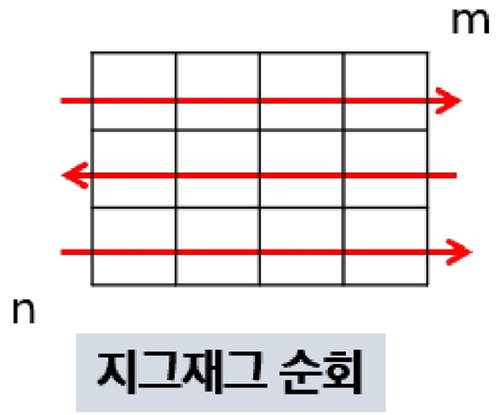
```
# i 행의 좌표
# j 열의 좌표

for j in range(m) :
    for i in range(n) :
        f(array[i][j]) # 필요한 연산 수행
```



<그림4 _SW문제해결 기본_List2_ 열 우선 순회>

지그재그 순회

```
# i 행의 좌표
# j 열의 좌표

for i in range(n) :
    for j in range(m) :
        f(array[i][j + (m-1-2*j) * (i%2)])
```

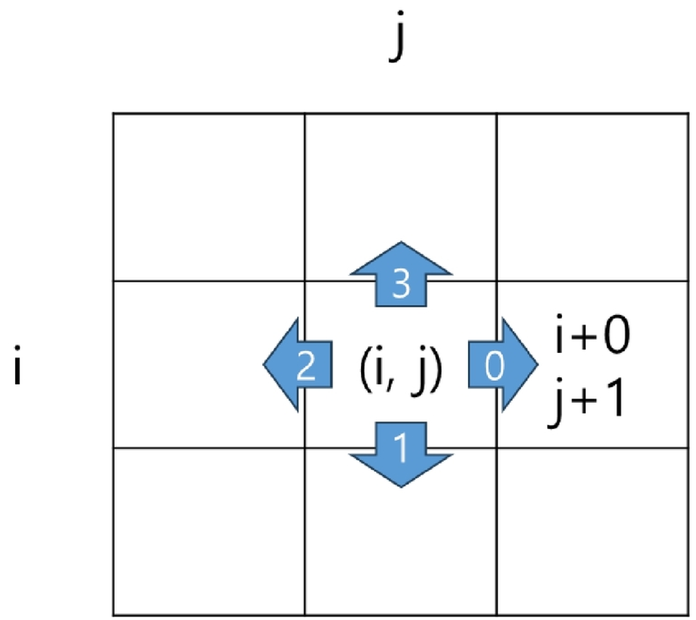


<그림5 _SW문제해결 기본_List2_ 지그재그 순회>

델타

델타를 활용한 2차원 배열 탐색

- 2차 배열의 한 좌표에서 4방향의 인접 배열 요소를 탐색하는 방법
- 인덱스 (i, j)인 칸의 상하좌우 칸 (ni, nj)



<그림6 _SW문제해결 기본_List2_ 델타>

```
di[] ← [0, 1, 0, -1] # 방향별로 더할 값
dj[] ← [1, 0, -1, 0]

for k : 0 -> 3
    ni <- i + di[k]
    nj <- j + dj[k]
```

델타를 활용한 2차원 배열 탐색 (1)

- 2차 배열의 한 좌표에서 4방향의 인접 배열 요소를 탐색하는 방법

```
arr[0…N-1][0…N-1] # NxN 배열
di[] ← [0, 1, 0, -1] #
dj[] ← [1, 0, -1, 0]
for i : 0 -> N-1
    for j : 0 -> N-1 :
        for d : 0 -> 3
            ni ← i + di[d]
            nj ← j + dj[d]
            if 0<=ni<N and 0<=nj<N   # 유효한 인덱스면
                f(arr[ni][nj])
```

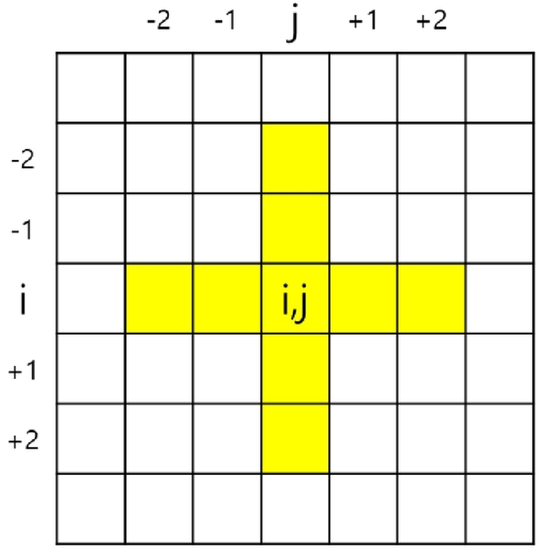
델타를 활용한 2차원 배열 탐색 (2)

- 2차 배열의 한 좌표에서 4방향의 인접 배열 요소를 탐색하는 방법

```
for i in range(N):
    for j in range(N):
        for di, dj in [[0,1],[1,0],[0,-1],[-1,0]]:
            ni, nj = i+di, j+dj
            …
```

델타 응용

- ex) NxN 배열에서 각 원소를 중심으로, 상하좌우 k칸의 합계 중 최댓값 (k=2)

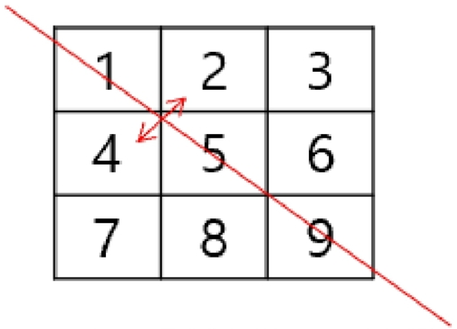


<그림7 _SW문제해결 기본_List2_ 델타 응용>

```
max_v = 0
for i in range(N):
    for j in range(N):
        s = arr[i][j]       # i,j를 중심으로
        for di, dj in [[0,1],[1,0],[0,-1],[-1,0]]:   # 각 방향
            for c in range(1, k+1):                   # 거리별
                ni, nj = i+di*c, j+dj*c
                if 0<=ni<N and 0<=nj<N:
                    s += arr[ni][nj]
        if max_v < s:
            max_v = s
```

전치 행렬

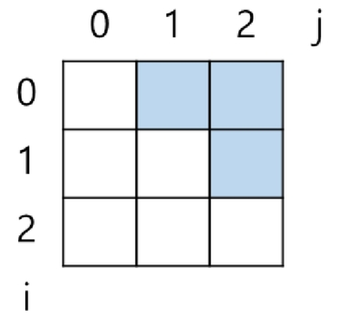
```
# i : 행의 좌표, 행의 크기 len(arr)
# j : 열의 좌표, 열의 크기 len(arr[0])
arr = [[1,2,3],[4,5,6],[7,8,9]] # 3*3 행렬

for i in range(3) :
    for j in range(3) :          # for j in range(i):인 경우
        if i < j :               # if문 필요 없음
            arr[i][j], arr[j][i] = arr[j][i], arr[i][j]
```



<그림8 _SW문제해결 기본_List2_ 전치 행렬>

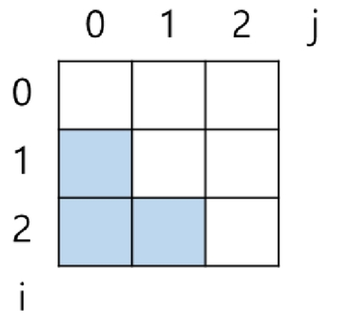
i, j의 크기에 따라 접근하는 원소 비교 (N x N 배열)

i < j



<그림9 _SW문제해결 기본_List2_i<j>

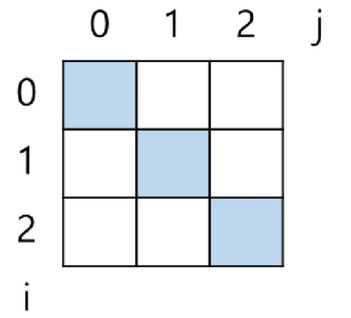
i > j



<그림10 _SW문제해결 기본_List2_i>j>

```
for i in range(N):
    for j in range(N):
        if          :
```

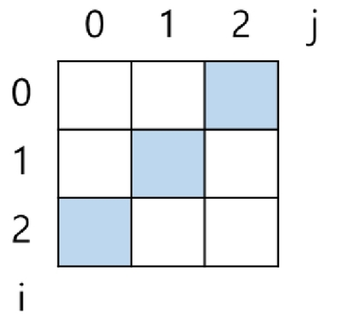
i == j



<그림11 _SW문제해결 기본_List2_i==j>

```
for i in range(N):
    f(arr[i][i])
```

N-1-i == j



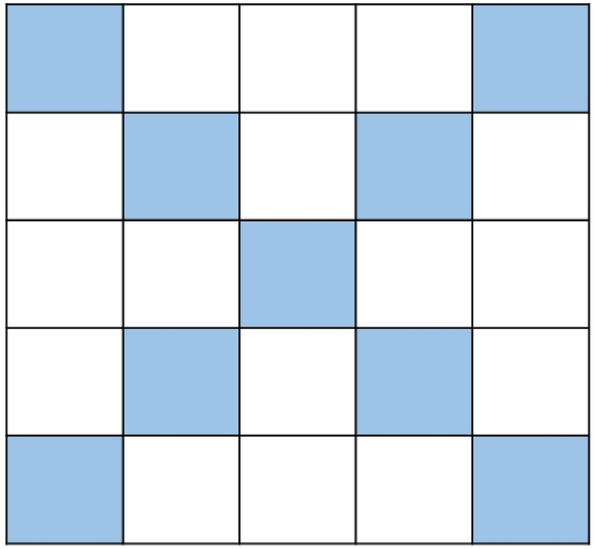
<그림12 _SW문제해결 기본_List2_N-1-i == j>

```
for i in range(N):
    f(arr[i][N-1-i])
```

연습 문제

연습 문제 1

- 5x5 2차원 배열에 25개의 숫자를 저장하고, 대각선 원소의 합을 구하시오.
- 대각선 원소는 다음과 같은 위치의 원소를 나타낸다.
- 입력파일 : 2_List2/연습문제1_in.txt



<그림13 _SW문제해결 기본_List2_연습문제1>

연습 문제 2

- 5x5 2차 배열에 25개의 숫자를 저장하고,
- 25개의 각 요소에 대해서 그 요소와 이웃한 요소와의 차의 절대값을 구하시오.
- 예를 들어 아래 그림에서 7의 이웃한 값은 2, 6, 8, 12 이며 차의 절대값의 합은 12 이다.

| 2 - 7 | + | 6 - 7 | + | 8 - 7 | + | 12 - 7 | = 12

...	2	...
6	7	8
...	12	...

<그림14 _SW문제해결 기본_List2_연습문제2>

- 25개의 요소에 대해서 모두 조사하여 총합을 구하시오.
- 벽에 있는 요소는 이웃한 요소가 없을 수 있음에 주의하시오.
  - 예를 들어 [0][0]은 이웃한 요소가 2개이다.
- 입력파일 : 2_List2/연습문제1_in.txt 사용

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

부분집합

부분집합 합 문제

- 유한 개의 정수로 이루어진 집합이 있을 때, 이 집합의 부분집합 중에서 조건에 맞는 경우를 찾는 문제
- 예) 부분집합의 원소를 모두 더한 값이 0이 되는 경우가 있는지를 알아내는 문제

※ 예시

예를 들어, [-7, -3, -2, 5, 8]라는 집합이 있을 때, [-3, -2, 5]는 이 집합의 부분집합이면서 (-3)+(-2)+5=0이므로 이 경우의 답은 참이 된다.

부분집합 생성하기

- 완전검색 기법으로 부분집합 합 문제 풀기
  - 우선 집합의 모든 부분집합을 생성한 후에 각 부분집합의 합을 계산합니다.

- 주어진 집합의 부분집합을 생성하는 방법 생각해보기

부분집합의 수

- 집합의 원소가 n개일 때, 공집합을 포함한 부분집합의 수는 2^n개
- 이는 각 원소를 부분집합에 포함시키거나 포함시키지 않는 2가지 경우를 모든 원소에 적용한 경우의 수와 같음
- 예) {1, 2, 3, 4}

→ 2 x 2 x 2 x 2 = 16가지

<그림15 _SW문제해결 기본_List2_부분집합의 수>

부분집합의 원소를 표현하는 방법

<그림16 _SW문제해결 기본_List2_부분집합의 원소 표현법>

```
arr[i] 원소가 부분집합에 포함되지 않으면 bit[i]==0
arr[i] 원소가 부분집합에 포함되면 bit[i]==1
```

각 원소가 부분집합에 포함되었는지를 loop를 이용하여 확인하고 부분집합을 생성하는 방법

```
bit = [0, 0, 0, 0]
for i in range(2) :
    bit[0] = i                  # 0번 원소
    for j in range(2) :
        bit[1] = j              # 1번 원소
        for k in range(2) :
            bit[2] = k          # 2번 원소
            for l in range(2) :
                bit[3] = l      # 3번 원소
                print_subset(bit)      # 생성된 부분집합 출력
```

이론

비트 연산자

- & :비트 단위로 AND 연산
- | : 비트 단위로 OR 연산
- << : 피연산자의 비트 열을 왼쪽으로 이동
- >> : 피연산자의 비트 열을 오른쪽으로 이동

<< 연산자 활용

- 1 << n : 2^n 즉, 원소가 n개일 경우의 모든 부분집합의 수를 의미

& 연산자 활용

- i & (1<<j) : i의 j번째 비트가 1인지 아닌지를 검사

비트 연산으로 부분집합을 생성하는 방법

- Python 코드 예

```
arr = [3,6,7,1,5,4]

n = len(arr)                # n : 원소의 개수

for i in range(1<<n):               # 1<<n : 부분 집합의 개수
    for j in range(n):              # 원소의 수만큼 비트를 비교함
        if i & (1<<j):              # i의 j번 비트가 1인경우
            print(arr[j], end=", ")             # j번 원소 출력
    print()
print()
```

연습문제3

- 10개의 정수를 입력받아 부분집합의 합이 0이 되는 것이 존재하는지 확인하기
- 입력 파일 : 2_List2/연습문제3_in.txt

※ 입력

```
-7 -5 2 3 8 -2 4 6 9 12
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

검색과 정렬

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

순차검색

검색의 정의

검색
Search

저장되어 있는 자료 중에서 원하는 항목을 찾는 작업
목적하는 탐색 키를 가진 항목을 찾는 것

※ 검색의 종류

순차 검색(sequential search), 이진 검색(binary search), 해쉬(hash)

탐색 키의 정의

탐색 키
Search Key

자료를 구별하여 인식할 수 있는 키

순차검색

- **일렬로 되어 있는 자료를 순서대로 검색하는 방법**
  - 가장 간단하고 직관적인 검색 방법입니다.
  - 배열이나 연결 리스트 등 순차구조로 구현된 자료구조에서 원하는 항목을 찾을 때 유용합니다.
  - 알고리즘이 단순하여 구현이 쉽지만, 검색 대상의 수가 많은 경우에는 수행시간이 급격히 증가하여 비효율적입니다.

- **정렬 여부에 따라**
  - 순차 검색 대상이 정렬되어 있지 않은 경우
  - 순차 검색 대상이 정렬되어 있는 경우

정렬되어 있지 않은 경우 순차검색

- **검색 과정**

① 첫 번째 원소부터 순서대로 검색 대상과 키 값이 같은 원소가 있는지 비교하며 찾는다.

② 키 값이 동일한 원소를 찾으면 그 원소의 인덱스를 반환한다.

③ 자료구조의 마지막에 이를 때까지 검색 대상을 찾지 못하면 검색 실패

정렬되어 있지 않은 경우에서 검색 성공

2를 검색하는 과정

① 4 ≠ 2	4	9	11	23	2	19	7
② 9 ≠ 2	4	9	11	23	2	19	7
③ 11 ≠ 2	4	9	11	23	2	19	7
④ 23 ≠ 2	4	9	11	23	2	19	7
⑤ 2 = 2	4	9	11	23	2	19	7

검색 성공!

<그림17 _SW문제해결 기본_List2_정렬되지 않은 자료의 순차 검색 성공>

정렬되어 있지 않은 경우에서 검색 실패

8을 검색하는 과정

① 4 ≠ 8	4	9	11	23	2	19	7
② 9 ≠ 8	4	9	11	23	2	19	7
③ 11 ≠ 8	4	9	11	23	2	19	7
④ 23 ≠ 8	4	9	11	23	2	19	7
⑤ 2 ≠ 8	4	9	11	23	2	19	7
⑥ 19 ≠ 8	4	9	11	23	2	19	7
⑦ 7 ≠ 8	4	9	11	23	2	19	7

검색 실패!

<그림18 _SW문제해결 기본_List2_정렬되지 않은 자료의 순차 검색 실패>

순차검색 대상이 정렬되어 있지 않은 경우

- **찾고자 하는 원소의 순서에 따라 비교 횟수가 결정됨**
  - 첫 번째 원소를 찾을 때는 1번 비교, 두 번째 원소를 찾을 때는 2번 비교..
  - 정렬되지 않은 자료에서의 순차 검색 성공 시 평균 비교 횟수
    = (1/n)*(1+2+3+…+n) = (n+1)/2
  - 검색 실패 시 마지막 원소까지 비교하므로 평균 비교 횟수 n
  - 시간 복잡도 : O(n)

구현 예

```
def sequential_search(a, n, key):
    i = 0
    while i < n and a[i] != key:
        i += 1
    if i < n:
        return i
    else:
        return -1
```

```
def sequential_search(a, n, key):
    for i in range(n):
        if a[i] == key:
            return i
    return -1
```

순차검색 대상이 정렬되어 있는 경우

- **검색 과정**

① 자료가 오름차순으로 정렬된 상태에서 검색을 실시한다.

② 자료를 순차적으로 검색하면서 키 값을 비교하여, 원소의 키 값이 검색 대상의 키 값보다 크면 찾는 원소가 없다는 것이므로 더 이상 검색하지 않고 검색을 종료한다.

정렬되어 있는 경우에서 검색 성공

11을 검색하는 과정

① 2 < 11

2	4	7	9	11	19	23

② 4 < 11

2	4	7	9	11	19	23

③ 7 < 11

2	4	7	9	11	19	23

④ 9 < 11

2	4	7	9	11	19	23

⑤ 11 = 11

2	4	7	9	11	19	23

검색 성공!

<그림19 _SW문제해결 기본_List2_정렬된 자료의 순차 검색 성공>

정렬되어 있는 경우에서의 검색 실패

10을 검색하는 과정

① 2 < 10	**2**	4	7	9	11	19	23
② 4 < 10	2	**4**	7	9	11	19	23
③ 7 < 10	2	4	**7**	9	11	19	23
④ 9 < 10	2	4	7	**9**	11	19	23
⑤ 11 > 10	2	4	7	9	**11**	19	23

검색 실패!

<그림20 _SW문제해결 기본_List2_정렬된 자료의 순차 검색 실패>

정렬되어 있는 경우

- **찾고자 하는 원소의 순서에 따라 비교 횟수가 결정됨**
  - 정렬이 되어있으므로, 검색 실패를 반환하는 경우에도 평균 비교 횟수가 반으로 줄어든다.
    (n+1)/2
  - 시간 복잡도 : O(n)

구현 예

```
def sequential_search2(a, n, key):
    i = 0
    while i < n and a[i] < key:
        i += 1
    if i < n and a[i] == key:
        return i
    else:
        return -1
```

```
def sequential_search2(a, n, key):
    for i in range(n):
        if a[i] == key:
            return i
        elif a[i] > key:
            return -1
    return -1
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

이진 검색

이진 검색 (Binary Search)

- **자료의 가운데에 있는 항목의 키 값과 비교하여 다음 검색의 위치를 결정하고 검색을 계속 진행하는 방법**
  - 목적 키를 찾을 때까지 이진 검색을 순환적으로 반복 수행함으로써 검색 범위를 반으로 줄여가면서 보다 빠르게 검색을 수행합니다.
- **이진 검색을 하기 위해서는 자료가 정렬된 상태여야 함**

검색 과정

① 자료의 중앙에 있는 원소를 고른다.

② 중앙 원소의 값과 찾고자 하는 목표 값을 비교한다.

③ 목표 값이 중앙 원소의 값보다 작으면 자료의 왼쪽 반에 대해서 새로 검색을 수행하고, 크다면 자료의 오른쪽 반에 대해서 새로 검색을 수행한다.

④ 찾고자 하는 값을 찾을 때까지 ①~③의 과정을 반복한다.

이진 검색으로 7을 찾는 경우

<그림21 _SW문제해결 기본_List2_이진 검색 성공>

이진 검색으로 20을 찾는 경우

<그림22 _SW문제해결 기본_List2_이진 검색 실패>

이진 검색 알고리즘

- **구현**
  - 검색 범위의 시작점과 종료점을 이용하여 검색을 반복 수행합니다.
  - 이진 검색의 경우, 자료에 삽입이나 삭제가 발생했을 때 배열의 상태를 항상 정렬 상태로 유지하는 추가 작업이 필요합니다.

이진 검색 알고리즘 구현

```
def binary_search(a, N, key): # key를 찾으면 인덱스, 실패하면 -1 반환
        start = 0
        end = N-1
        while start <= end :
                middle = (start + end)//2
                if a[middle] == key :   # 검색 성공
                        return middle
                elif a[middle] > key :  # 찾는 값보다 크면
                        end = middle – 1                # 왼쪽 구간 선택
                else :                                  # 찾는 값보다 작으면
                        start = middle + 1              # 오른쪽 구간 선택
        return -1                                       # 검색 실패
```

이진 검색 알고리즘 구현

- **재귀 함수 이용**
  - 아래와 같이 재귀 함수를 이용하여 이진 검색을 구현할 수도 있습니다.
  - 재귀 함수에 대해서는 나중에 더 자세히 배우도록 합니다.

```
def binary_search2(a, low, high, key) :
    if low > high : # 검색 실패
        return False
    else :
        middle = (low + high) // 2
        if key == a[middle] : # 검색 성공
            return True
        elif key < a[middle] :
            return binary_search2(a, low, middle-1, key)
        elif a[middle] < key :
            return binary_search2(a, middle+1, high, key)
```

인덱스

- **원본 데이터 배열과 별개로, 배열 인덱스를 추가한 예**
  - 원본 데이터에 데이터가 삽입될 경우 상대적으로 크기가 작은 인덱스 배열을 정렬하기 때문에 속도가 빠릅니다.

array index	name	original index
0	Barbara	1
1	Florence	2
2	Melissa	0
3	Shara	3

이름 인덱스 배열

array index	id	original index
0	1041	0
1	7334	2
2	19467	1
3	27500	3

array index	id	name	number	…
0	1041	Melissa	428-849-0471	…
1	19467	barbara	672-511-7155	…
2	7334	Florence	586-122-3241	…
3	27500	shara	459-729-8167	…
…	…	…	…	…

원본 데이터 배열

<그림23 _SW문제해결 기본_List2_인덱스>

이론

인덱스

- 인덱스라는 용어는 Database에서 유래했으며, 테이블에 대한 동작 속도를 높여주는 자료 구조
- Database 분야가 아닌 곳에서는 Lookup table 이라고도 함
- 인덱스를 저장하는데 필요한 디스크 공간은 보통 테이블을 저장하는데 필요한 디스크 공간보다 작음

 - 보통 인덱스는 키-필드만 갖고 있고, 테이블의 다른 세부 항목들은 갖고 있지 않기 때문입니다.

이론

인덱스

- 대량의 데이터를 매번 정렬하면, 프로그램의 반응이 느려짐
- 이러한 대량 데이터의 성능 저하 문제를 해결하기 위해 배열 인덱스를 사용

※ 데이터베이스 인덱스는 이진 탐색 트리 구조로 되어있습니다.

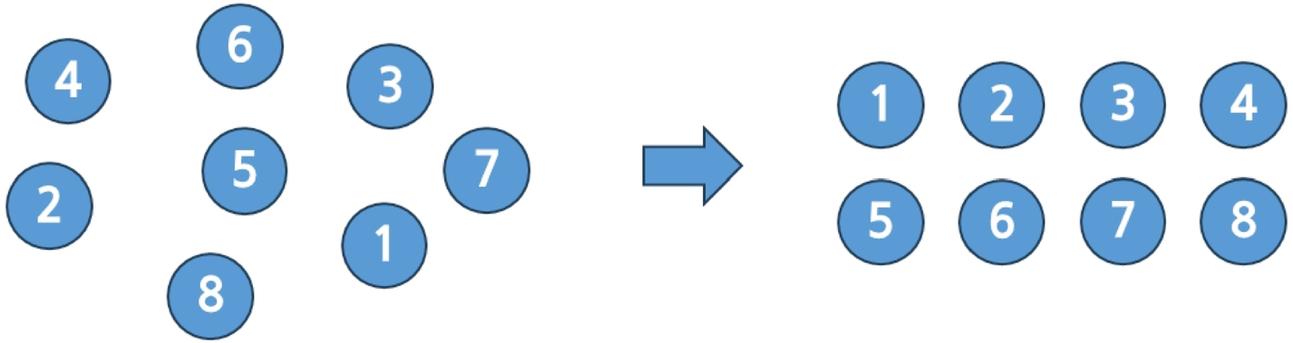
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

선택 정렬

선택 정렬

- **포켓볼 순서대로 정렬하기**
  - 흩어진 당구공을 그림처럼 정리한다고 한다면 어떻게 할까요?



<그림24 _SW문제해결 기본_List2_포켓볼 정렬하기>

많은 사람들은 당구대 위에 있는 공에서 가장 작은 숫자의 공부터 골라서 차례대로 정리할 것입니다.
이것이 바로 선택 정렬입니다.

선택 정렬

- 주어진 자료들 중 가장 작은 값의 원소부터 차례대로 선택하여 위치를 교환하는 방식 (오름차순의 경우)

정렬 과정

- 주어진 리스트 중에서 최솟값을 찾기
- 그 값을 리스트의 맨 앞에 위치한 값과 교환
- 맨 처음 위치를 제외한 나머지 리스트를 대상으로 위의 과정을 반복

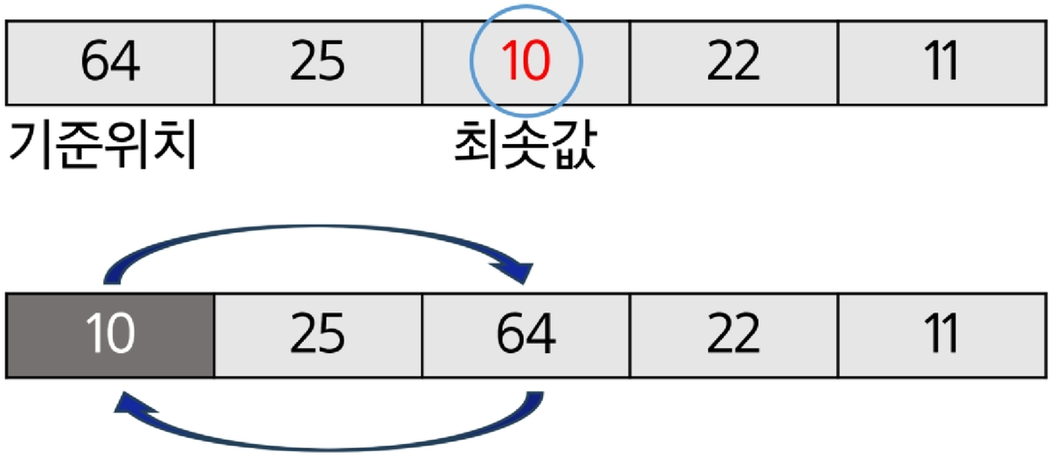
시간 복잡도

- $O(n^2)$

선택 정렬 과정

① 주어진 리스트에서 최솟값을 찾는다.

② 리스트의 맨 앞에 위치한 값과 교환한다.



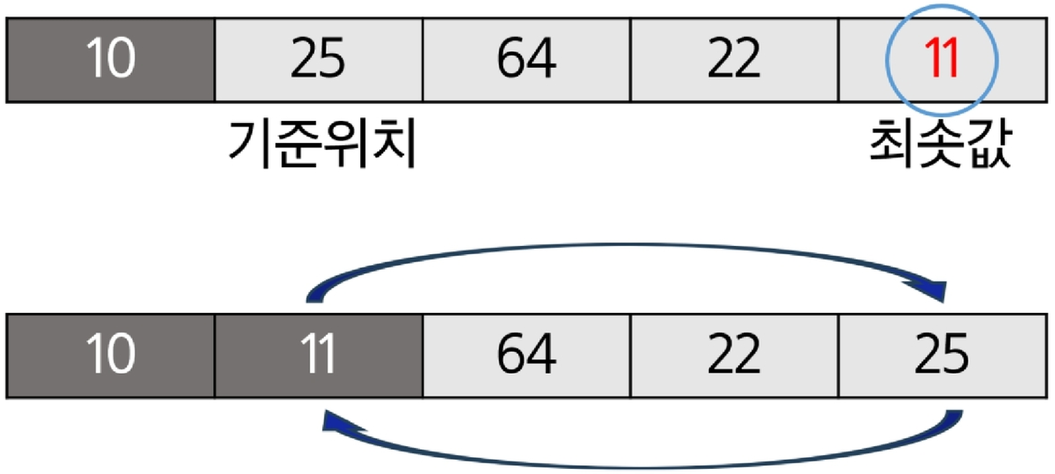
정렬된 원소 미정렬 원소

<그림25 _SW문제해결 기본_List2_선택 정렬 첫 구간>

선택 정렬 과정

③ 미정렬 리스트에서 최솟값을 찾는다.

④ 리스트의 맨 앞에 위치한 값과 교환한다.



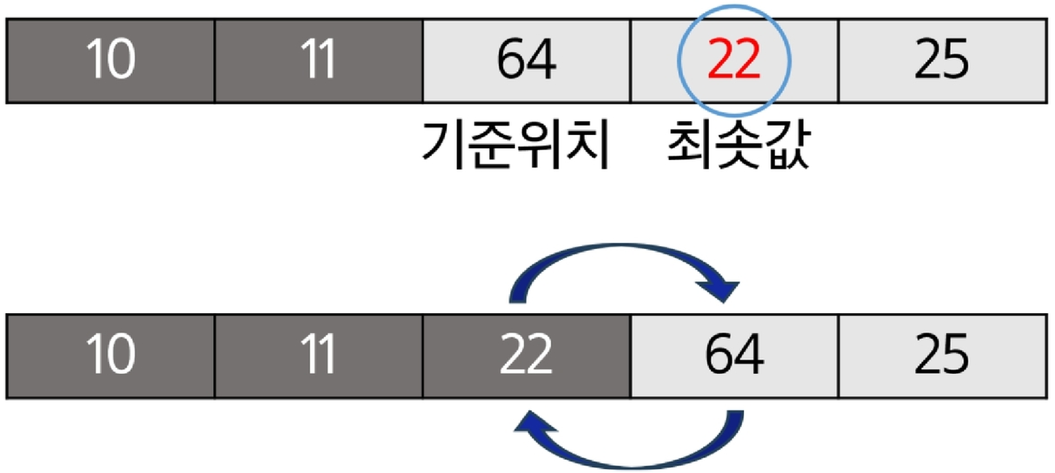
정렬된 원소 미정렬 원소

<그림26 _SW문제해결 기본_List2_선택 정렬 두 번째 구간>

선택 정렬 과정

⑤ 미정렬 리스트에서 최솟값을 찾는다.

⑥ 리스트의 맨 앞에 위치한 값과 교환한다.

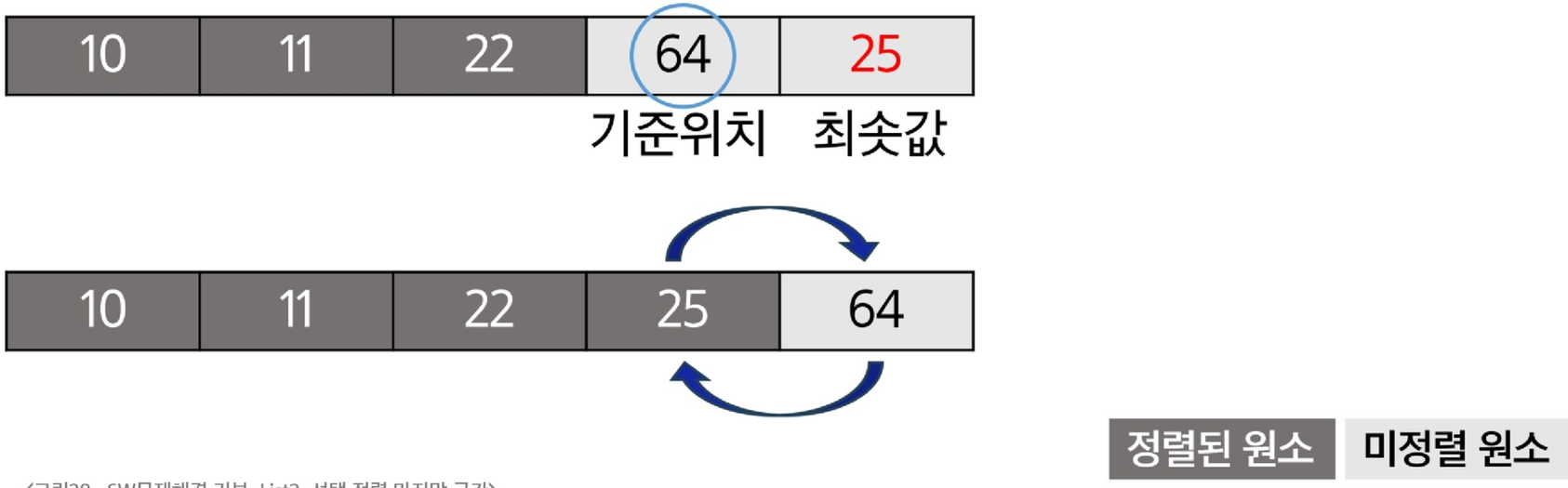


<그림27 _SW문제해결 기본_List2_선택 정렬 세 번째 구간>

선택 정렬 과정

⑦ 미정렬 리스트에서 최솟값을 찾는다.

⑧ 리스트의 맨 앞에 위치한 값과 교환한다.



<그림28 _SW문제해결 기본_List2_선택 정렬 마지막 구간>

- 미정렬 원소가 하나 남은 상황에서는 마지막 원소가 가장 큰 값을 갖게 되므로, 실행을 종료하고 선택 정렬이 완료된다.

선택 정렬 알고리즘

```
selection_sort(a[], n):
        for i : 0 -> n-2
                a[i],…,a[n-1] 원소 중 최솟값 a[k] 찾기
                a[i]와 a[k] 교환
```

선택 정렬

```
def selection_sort(a, N) :
    for i in range(N-1) :   # 정렬 구간의 시작 인덱스
        min_idx = i             # 첫 원소를 최솟값으로 가정
        for j in range(i+1, N) :
            if a[min_idx] > a[j] : # 최솟값의 인덱스 갱신
                min_idx = j
        a[i], a[min_idx] = a[min_idx], a[i] # 구간 최솟값을 구간 맨 앞으로
```

정렬 알고리즘 비교

알고리즘	평균 수행시간	최악 수행시간	알고리즘 기법	비고
버블 정렬	$O(n^2)$	$O(n^2)$	비교와 교환	코딩이 가장 손쉽다.
카운팅 정렬	O(n+k)	O(n+k)	비교환 방식	k가 비교적 작을 때만 가능하다.
선택 정렬	$O(n^2)$	$O(n^2)$	비교와 교환	교환의 회수가 버블, 삽입정렬보다 작다.
퀵 정렬	O(n log n)	$O(n^2)$	분할 정복	최악의 경우 O(n2) 이지만, 평균적으로는 가장 빠르다.
삽입 정렬	$O(n^2)$	$O(n^2)$	비교와 교환	n의 개수가 작을 때 효과적이다.
병합 정렬	O(n log n)	O(n log n)	분할 정복	연결리스트의 경우 가장 효율적인 방식이다.

〈표1 _SW문제해결 기본_List2_정렬 알고리즘 비교〉

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

셀렉션 알고리즘

셀렉션 알고리즘

- 저장되어 있는 자료로부터 k번째로 큰 혹은 작은 원소를 찾는 방법을 셀렉션 알고리즘이라 한다.
  - 최솟값, 최댓값 혹은 중간값을 찾는 알고리즘을 의미하기도 한다.

선택 과정

- 셀렉션은 아래와 같은 과정을 통해 이루어진다.
  - 정렬 알고리즘을 이용하여 자료 정렬하기
  - 원하는 순서에 있는 원소 가져오기

K번째로 작은 원소를 찾는 알고리즘

- 1번부터 k번째까지 작은 원소들을 찾아 배열의 앞쪽으로 이동시키고, 배열의 k번째를 반환한다.
- k가 비교적 작을 때 유용하며 O(kn)의 수행시간을 필요로 한다.

```
def select(arr, k) :
    for i in range(0, k) :
        min_index = i
        for j in range (i+1, len(arr)):
           if arr[min_index] > arr[j] :
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr[k-1]
```

실습

2차원 List

- 4836. 색칠하기

부분집합

- 4837. 부분집합의 합

탐색과 정렬

- 4839. 이진탐색
- 4843. 특별한 정렬

추가 연습

- 2001. 파리퇴치
- 1979. 어디에 단어가 들어갈 수 있을까
- 1954. 달팽이 숫자

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 다음 중 2차원 배열의 선언으로 올바른 것은?

a) arr = [0, 1, 2, 3]
b) arr = [[1, 2, 3], [4, 5, 6]]
c) arr = (1, 2), (3, 4)
d) arr = [[1, 2], 3, 4]

2 n개의 원소로 이루어진 집합의 부분집합의 개수는 몇 개인가?

a) n
b) n**2
c) 2**n
d) n!

3 델타 탐색에서 상하좌우를 나타내는 방향 배열로 알맞은 것은?

a) di = [1, -1, 0, 0], dj = [0, 0, 1, -1]
b) di = [0, 1, 0, -1], dj = [1, 0, -1, 0]
c) di = [-1, 1, -1, 1], dj = [1, -1, -1, 1]
d) di = [1, 1, 1, 1], dj = [0, 1, -1, 0]

4 다음 중 비트 연산을 통해 부분집합을 확인하는 코드에서 if I & (1 << j) 가 의미하는 것은?

a) i가 j보다 작으면
b) i의 j번째 비트가 1이면
c) i가 j보다 크면
d) j가 i의 인덱스일 때

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 다음 중 순차 검색(정렬되지 않은 배열)의 시간 복잡도는?

a) O(log n)
b) O(1)
c) O(n)
d) O(n log n)

6 선택 정렬의 특징으로 옳은 것은?

a) 항상 O(log n)으로 동작한다
b) 선택된 요소만 정렬하고 나머지는 무시한다
c) 정렬 중 교환 횟수가 비교적 적다
d) 가장 큰 수부터 앞으로 보낸다

7 이진 검색을 사용할 수 없는 경우는?

a) 배열이 정렬되지 않은 경우
b) 배열의 길이가 짝수인 경우
c) 배열의 길이가 홀수인 경우
d) 값이 음수인 경우

8 다음 중 전치(transpose) 행렬에 대한 설명으로 옳은 것은?

a) 각 행의 평균을 구한다
b) 대각선 방향으로만 값을 바꾼다
c) 행과 열의 위치를 바꾸는 연산이다
d) 모든 값에 -1을 곱한다

확인문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 다음 중 '델타 탐색'에서 고려해야 할 조건으로 옳은 것은?

a) 인접한 모든 원소가 0인지 확인
b) 인덱스가 유효한 범위 내에 있는지 확인
c) 배열이 오름차순인지 확인
d) 부분집합의 개수를 세야 한다

10 다음 중 selection 알고리즘의 설명으로 적절하지 않은 것은?

a) k번째로 작은 원소를 찾을 때 사용된다
b) 모든 요소를 정렬한 뒤 원하는 순서를 가져온다
c) 일반적으로 O(log n)의 시간 복잡도를 가진다
d) 최솟값, 최댓값, 중간값 탐색에 사용된다

확인문제

정답 및 해설

1. b) arr = [[1, 2, 3], [4, 5, 6]] 2. c) 2**n 3. b) di = [0, 1, 0, -1], dj = [1, 0, -1, 0] 4. b) i의 j번째 비트가 1이면 5. c) O(n) 6. c) 정렬 중 교환 횟수가 비교적 적다 7. a) 8. c) 행과 열의 위치를 바꾸는 연산이다 9. b) 인덱스가 유효한 범위 내에 있는지 확인 10. c) 일반적으로 O(log n)의 시간 복잡도를 가진다

1.2차원 배열은 리스트의 리스트 형태로 선언되며, [[1, 2, 3], [4, 5, 6]]은 2행 3열 구조로 올바릅니다.

3. 각 원소가 포함되거나 포함되지 않는 두 가지 경우가 있으므로, 전체 부분집합의 수는 2**n입니다.

2. 상하좌우 순서의 방향 벡터는 각각 (0,1), (1,0), (0,-1), (-1,0) 입니다.

4. (1 << j)는 j번째 비트가 1인 수이고, i & (1<<j)는 i의 j번째 비트가 1인지 확인하는 표현입니다.

5. 정렬되지 않은 배열에서는 순차적으로 모든 요소를 비교하므로 시간 복잡도는 O(n)입니다.

6. 선택 정렬은 전체 정렬 알고리즘 중 교환 횟수가 비교적 적습니다.

7. 이진 검색은 반드시 정렬된 배열에서만 수행할 수 있습니다. 정렬되지 않은 배열에서는 동작하지 않습니다.

8. 전치 행렬은 i행 j열의 원소를 j행 i열로 바꾸는 연산입니다.

9. 델타 탐색은 상하좌우를 탐색하므로 배열의 범위를 벗어나지 않도록 인덱스 유효성 검사를 꼭 해야 합니다.

10. selection 알고리즘은 O(kn)의 시간복잡도를 가집니다.

활동정리

핵심키워드

개념	설명
2차원 배열	리스트의 리스트로 구성된 다차원 배열 구조
델타 탐색	상하좌우 인접 요소를 탐색할 때 사용하는 방향 벡터
부분집합	주어진 집합에서 원소를 선택하는 모든 조합을 구하는 방법
순차 검색	데이터를 처음부터 끝까지 하나씩 비교하며 찾는 검색 방식
이진 검색	정렬된 배열에서 중간값 기준으로 절반씩 줄이며 탐색하는 검색 방식
선택 정렬	가장 작은 값을 앞에서부터 하나씩 선택하여 정렬하는 방식

<표2 _SW문제해결 기본_List2_핵심 키워드>

활동정리

오늘 공부한 내용 요약 및 정리

- 2차원 배열이란?
  - 리스트 안에 리스트가 있는 구조로, 행과 열로 구성됨
  - arr = [[1, 2], [3, 4]] 형태

- 델타를 활용한 인접 탐색
  - 상하좌우 인접 원소를 탐색할 때 방향 벡터 사용
  - di = [0, 1, 0, -1], dj = [1, 0, -1, 0]

- 부분집합 생성 방법
  - n개의 원소가 있다면 부분집합의 수는 2^n개
  - 중첩 반복문 또는 비트 연산자를 이용해 생성 가능

활동정리

오늘 공부한 내용 요약 및 정리

- 순차 검색과 이진 검색
  - 순차 검색 : 정렬 X, 앞에서부터 차례대로 비교
  - 이진 검색 : 정렬 O, 중간값 기준으로 범위를 반으로 줄이며 탐색

- 선택 정렬(Selection Sort)
  - 가장 작은 값을 선택해 앞에서부터 정렬
  - 시간 복잡도 $O(n^2)$

활동정리

'리스트 안에서 기준보다 큰 값을 찾아 개수를 세어 봅시다!'

1. 숫자들이 들어 있는 리스트를 변수 [ ] 에 저장합니다.
2. 반복문을 이용해 리스트의 값을 하나씩 꺼내면서 [ ] 조건을 사용해 50보다 큰 값만 골라냅니다.
3. 조건을 만족하면 [ ] 값을 1씩 증가시켜 개수를 셉니다.

활동정리

'리스트 안에서 기준보다 큰 값을 찾아 개수를 세어 봅시다!'

1. 숫자들이 들어 있는 리스트를 변수 numbers에 저장합니다.
2. 반복문을 이용해 리스트의 값을 하나씩 꺼내면서 if num > 50 조건을 사용해 50보다 큰 값만 골라냅니다.
3. 조건을 만족하면 count 값을 1씩 증가시켜 개수를 셉니다.

```
numbers = [45, 67, 32, 90, 51]
count = 0
for num in numbers:
    if num > 50:
        count += 1
print("50보다 큰 값의 개수:", count)
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다